Frontend Architecture: Next.js Game Portal

ECOSYSTEM HUB & CLIENT HANDSHAKE PROTOCOL SPECIFICATION

This document details the frontend implementation architecture for the central **Kapp Studio Portal** built using the **Next.js App Router** framework. It handles unified user state delivery, dynamic route processing, and cross-origin security pipelines to host individual standalone web or electron game contexts.

1. Project Folder Topography (Next.js App Router)

The system leverages Next.js file-system routing to map page components dynamically while keeping shared platform modules organized:

```
kapp-studio-portal/
├── public/
│   └── assets/               # High-res studio banners, sound maps, game previews
├── src/
│   ├── app/                  # Application Route Matrix
│   │   ├── layout.js         # Shell configuration (Global Navigation, Theme contexts)
│   │   ├── page.js           # Core Player Dashboard (Aggregation & Analytics)
│   │   ├── leaderboards/
│   │   │   └── page.js       # Global scoreboard tables (Server Components)
│   │   └── games/
│   │       ├── page.js       # Unified Grid Catalogue (The 4 game cards)
│   │       └── [gameId]/
│   │           └── page.js   # Dynamic Game Wrapper Frame (Client Sandbox Container)
│   ├── components/           # Reusable Presentation Components
│   │   ├── GameCard.js       # Preview launcher widget with telemetry stats
│   │   └── SyncIndicator.js  # Live WebSocket/Cloud persistence status indicator
│   └── context/
│   │   └── AuthContext.js    # Client-side session container (JWT management)
```

2. Cross-Origin Handshake Engine (Client Component)

Since the browser's mounting environment requires runtime execution steps, the dynamic route `src/app/games/[gameId]/page.js` explicitly initializes using the `"use client"` instruction block to securely deliver state payloads downwards to nested iframes:

```
"use client";

import { useContext, useRef, useEffect } from 'react';
import { useParams, useRouter } from 'next/navigation';
import { AuthContext } from '@/context/AuthContext';

export default function GameRunner() {
  const { gameId } = useParams();
  const router = useRouter();
  const { user, token } = useContext(AuthContext);
  const iframeRef = useRef(null);

  const gameUrls = {
    'typing-math': process.env.NEXT_PUBLIC_GAME_TYPING_MATH_URL || 'http://localhost:
5174',
    'robot-quest': process.env.NEXT_PUBLIC_GAME_ROBOT_QUEST_URL || 'http://localhost:5175'
  };

  const targetGameUrl = gameUrls[gameId];

  useEffect(() => {
    const handleFrameLoad = () => {
      if (iframeRef.current && token) {
        const payload = {
          source: 'kapp-studio-portal',
          action: 'SYNC_AUTH_SESSION',
          user: { id: user.id, username: user.username },
          token: token
        };
        iframeRef.current.contentWindow.postMessage(payload, targetGameUrl);
      }
    };

    const frame = iframeRef.current;
    if (frame) {
      frame.addEventListener('load', handleFrameLoad);
    }
    return () => {
      if (frame) frame.removeEventListener('load', handleFrameLoad);
    };
  }, [token, targetGameUrl, user]);

  if (!targetGameUrl) return
Game Route Configuration Not Located.
;

  return (
```

```
      |
    </div>
  );
}</code></pre>

    <h2>3. Core Architectural Advantages</h2>
    <ul>
        <li><strong>Server-Side Rendered (SSR) Scoring Layouts:</strong> Leaderboards and competitive stats pages fetch high-score properties directly on the server during request generation, avoiding screen flashes or loading spinners.</li>
        <li><strong>Automated Banner Optimization:</strong> Custom game banners and logos are fully processed using Next.js <code>next/image</code> components to scale performance cleanly.</li>
        <li><strong>Uniform Configuration Routing:</strong> Environment-specific target paths are handled completely by Next.js configuration maps.</li>
    </ul>
</body>
</html>
```